UNIVERSIDADE DE SÃO PAULO

Instituto de Matemática, Estatística e Ciência da Computação

MAC0216 — Técnicas de Programação I

Biblioteca de ML

Biblioteca Modular em C para Aprendizado de Máquina

Relatório Técnico — Trabalho Prático 2

Integrantes

Felipe Brancalhão Curcio	16985106
Jaime Cavalcanti Guedes	16875204
João Pedro Silva Amadio	13672533
Murilo Santos de Castro	17015895

São Paulo

2026

Sumário

1 Introdução **1**

2 Makefile **1**

2.1 Variáveis de diretório 1

2.2 Variáveis de compilação 1

2.3 Descoberta dos arquivos-fonte 1

2.4 Criação de diretórios via Makefile 2

2.5 Compilação dos arquivos via Makefile 2

3 Testes **2**

3.1 Testes do módulo **CLI** 2

1 Introdução

Este relatório descreve o projeto e a implementação da `Biblioteca de ML`, uma biblioteca modular escrita em linguagem C e voltada para a área de aprendizado de máquina (*Machine Learning*), acompanhada de uma aplicação cliente de linha de comando. O código-fonte está disponível no repositório TP2.

2 Makefile

2.1 Variáveis de diretório

Para evitar problemas na compilação do código devido a mudanças na estrutura do projeto, foram definidas as seguintes variáveis contendo os caminhos dos diretórios:

- *INCLUDE_DIR*: diretório que contém os arquivos .h
- *APP_DIR*: diretório que contém a main.c
- *SRC_DIR*: diretório que contém as implementações dos arquivos .h
- *TESTS_DIR*: diretório que contém os testes lógicos
- *BIN_DIR*: diretório que contém os arquivos .o
- *BUILD_DIR*: diretório que contém os arquivos executáveis

2.2 Variáveis de compilação

- *CC*: compilador utilizado (Default = gcc)
- *CFLAGS*: *flags* de compilação (Default = -Wall -Wextra -g -std=c11)
- *CPPFLAGS*: onde buscar os arquivos .h

A fim de visualizar os valores recebidos por cada uma destas variáveis, foi definida a função auxiliar *exibe_variaveis*

2.3 Descoberta dos arquivos-fonte

Para que o Makefile consiga encontrar de maneira independente os arquivos .c e seguir um modelo de nomenclatura estabelecido a fim de criar os objetos, foi utilizada uma combinação das funções *wildcard* e *patsubst*.

Assim, o Makefile procura nos diretórios *src/* e *app/* por arquivos .c e salva seus nomes em 2 listas diferentes, utilizando como critério o diretório de origem do arquivo.

Ou seja, ao fim desta etapa tem-se uma lista com os arquivos .c vindos de *src/* e outra com aqueles vindos de *app/*.

Com isso, o Makefile separa o nome de cada arquivo de seu prefixo e sufixo, utilizando-o para nomear o arquivo objeto, já na pasta correspondente (*build/*).

2.4 Criação de diretórios via Makefile

Como os diretórios *build/* e *bin/* não estão sendo versionados, a fim de deixar o histórico do github mais limpo, é necessário garantir que ambos existam antes de rodar algo como *make run*.

Por isso, criou-se um comando que, utilizando as variáveis definidas nas seções anteriores, cria tais subdiretórios.

2.5 Compilação dos arquivos via Makefile

Foram criadas duas regras para a compilação dos arquivos .c em arquivos .o - ambas utilizam a mesma lógica - apenas diferem no local onde procuram os arquivos.

Para gerar um arquivo cujo nome é especificado no momento de chamada do comando, o Makefile primeiro procura pelo .c correspondente no diretório *src/*. Além disso, é verificada a existência do subdiretório *build/* que, caso seja inexistente, é criado via regra anterior. Se o arquivo não existir em *src/*, então o Makefile procura em *app/* e, somente em falha deste segundo, retorna erro.

3 Testes

3.1 Testes do módulo CLI

Os testes deste módulo foram divididos em três categorias:

- Testes unitários com valores fixos:
  O intuito deste tipo de teste é testar a função *validar_escolha_funcionalidade* em casos de inputs inválidos. Sua comparação é feita diretamente entre o retorno da função e o valor esperado.

- Testes unitários com entrada redirecionada:
  Testar a função *parsear_escolha_funcionalidade_usuario* em casos de buffer de entrada alterados. Para simular o usuário, foi utilizado um arquivo .entrada. Sua comparação é feita diretamente entre o retorno da função e o valor esperado.

- Testes de saída:
  Testar a função *cli_run* em casos de seleções de funcionalidades subsequentes. Redireciona os printfs da função para um arquivo de saída e depois compara com os valores esperados usando *diff -u*.